

1. Program ima dve paralelne porcije (dela) i jednu sekvencijalnu. Dve paralelne porcije čine, redom, 30% i 40% programa. Ako se program izvršava na 4-procesorskom sistemu, ubrzanje dva paralelna dela iznosi, redom, 3 i 4. Koliko je ukupno ubrzanje programa?
2. Razmotrimo multiprocesorski sistem sa 3 procesora P1, P2 i P3. Pretpostavimo da su povezani ili zajedničkom magistralom, ili linearnim vezom ili prstenom. U trenutku 1, sva tri procesora žele da pošalju poruku: P1 treba da pošalje poruku ka P2, P2 ka P3 i P3 ka P1. Pretpostavimo da je svaka veza iste brzine. Koliko dugo traje ova komunikacija za svaku od mreža i zašto?
3. a) Objasniti GCD test za detekciju loop carry zavisnosti.
b) Korišćenjem GCD test utvrditi da li u sledećem gnezdu petlji postoji loop carry zavisnost.

```
for (i=0; i<p; i++)  
  for (j=1; j<q; j++)  
    for (k=1; k<r; k++)  
      a(4*i+2*j+1) = ... A(6*i+2*k+4);
```

4. Sledeća pitanja se odnose na programe označene sa Code 1A i Code 1B.
Koja schedule odredba se treba upotrebiti u Code 1A da bi se postiglo najmanji disbalans u opterećenju. Nemojte uzimati u obzir cenu planiranja (scheduling). Dati kratko obrazloženje za odabrani odgovor.
(a) schedule (static)
(b) schedule (static, k) za malo k
(c) schedule (dynamic, k), za malo k
(d) a and b
(e) b and c

Koja schedule odredba se treba upotrebiti u Code 1B da bi se postigao dobar balans opterećenja i mala cena planiranja (low overhead) Dati kratko obrazloženje za odabrani odgovor.

- (a) schedule (static)
- (b) schedule (static, k) za malo k
- (c) schedule (dynamic, k), za malo k
- (d) a and b
- (e) b and c

```
int main (int argc, char *argv[]) {
```

```
  int i;  
  int j;  
  int n = 1000;  
  float v[n], a[n][n], b[n];  
  
  #pragma omp parallel for private(j)  
  for (i=0; i < n; i++) {  
    for (j=i; j < n; j++) {  
      for (j=i; j < n; j++) {  
        v[i] += a[i][j]*b[i];  
      }  
    }  
  }  
}
```

Code 1A

```
int main (int argc, char *argv[]) {
```

```
  int i;  
  int j;  
  int n = 1000;  
  float v[n], a[n][n], b[n];  
  
  #pragma omp parallel for private(j)  
  for (i=0; i < n; i++) {  
    for (j=0; j < n; j++) {  
      v[i] += a[i][j]*b[i];  
    }  
  }  
}
```

Code 1B

5. Dvo procesorski sistem koristi Write once protokol za održavanje keš koherencije. Promenljive X i Y preslikavaju se u isti keš blok. Inicijalno $X=0$ i $Y=0$. Popuniti donju tabelu za sledeći scenario:

Akcija	Stanje keša C1	Stanje keša C2	Komanda konzistencije	Vrednosti X i Y u GM
P1: $X=1$				
P2: $Y=1$				
P1: $X=2$				

- 6.
- Napisati MPI program koji realizuje množenje matrice $A_{n \times s}$ i matrice $B_{s \times k}$, čime se dobija rezultujuća matrica C. Program pronalazi i prikazuje i proizvod svake vrste matrice B. Matrice A i B se inicijalizuju u master procesu. Master proces šalje svakom procesu celu matricu A. Matrica B je podeljena u blokove po kolonama (k je deljivo sa p) i to tako da proces P_i dobija kolone sa indeksima $i, i \bmod p = i$ tj. kolone sa indeksima $i, i+p, i+2p, \dots, i+k-p$. Predvideti da se slanje kolona matrice B svakom procesu obavlja odjednom i direktno iz matrice B. Svaki proces obavlja odgovarajuća izračunavanja i učestvuje u generisanju rezultata. Rezultate programa prikazati u proizvoljnom procesu. Zadatak rešiti korišćenjem grupnih operacija.
 - Napisati MPI program koji će omogućiti komunikaciju samo procesa na glavnoj dijagonali procesa u kvadratnoj mreži procesa kreiranjem komunikatora. Master proces (P0) svim procesima ove grupe šalje po jednu kolonu matrice A. Prikazati identifikatore procesa koji pripadaju novom komunikatoru, kao i proizvod primljene kolone u svakom procesu komunikatora.

7. Na programskom jeziku C/C++, korišćenjem CUDA tehnologije, napisati program koji proverava da li se korisnička lozinka L nalazi u bazi procurelih lozinki B, dužine $32 * n$ karaktera. Baza B je u memoriji upamćena kao niz karaktera, u kojem svaka lozinka zauzima tačno 32 bajta, pri čemu su kraće lozinke dopunjene nulama (0). Prikazati korisniku upozorenje o bezbednosnom riziku ukoliko je pronađeno poklapanje njegove lozinke. Prilikom implementacije obratiti pažnju na efikasnost paralelizacije i omogućiti pozivanje kernela za bazu lozinki proizvoljne veličine.

8. Data je petlja:

```

int h=0;
for(i=2; i<N; i++)
  for (j=0; j<M; j++)
  {
    h+= a[i][j];
    a[i][j] = a[i-2][j];
  }

```

- Da li je moguće izvršiti paralelizaciju date petlje bez modifikacija? Obrazložiti. Ukoliko postoje zavisnosti, navesti koje su i objasniti ih.
- Transformisati petlju tako da paralelizacija bude moguća i paralelizaciju izvršiti korišćenjem OpenMP.
- Razvijanjem petlje za konkretne inicijalizacije vrednosti M, N i niza atestirati sekvencijalno i paralelno rešenje i pokazati da je petlja korektno paralelizovana rešenjem pod b).